ALPE FORMACIÓN · SANTANDER

Edición 2026 · PHP 8.x

CURSO DE PHP · CLASE 16

Proyecto CRUD paso a paso

- Qué vamos a construir
- La estructura de carpetas
- Paso 1 · Configuración y conexión
- Paso 2 · La caja de herramientas
- Paso 3 · Las consultas, en su propio archivo
- Paso 4 · Cabecera y pie comunes
- Paso 5 · Listado con buscador y paginación
- Paso 6 · Alta de cursos
- Paso 7 · Edición con el formulario precargado
- Paso 8 · Borrado seguro
- Paso 9 · Login contra la base de datos
- Probad vuestra aplicación



Ramón Abramo

+25 años de experiencia docente · Santander, Cantabria

ALPE

FORMACIÓN PROFESIONAL

🧭 Contenido de la clase

Ha llegado el momento de juntar todas las piezas. Formularios, validación, sesiones, seguridad, `require_once`, PDO, transacciones... Todo lo que habéis aprendido desde la clase 7 lo vamos a usar hoy para construir **una aplicación completa**: el catálogo de cursos de la academia, con su listado, su buscador, sus formularios y su zona privada.

CRUD son las iniciales en inglés de las cuatro operaciones que hace casi cualquier aplicación con datos: *Create* (crear), *Read* (leer), *Update* (actualizar) y *Delete* (borrar). Si sabéis hacer un CRUD bien hecho, sabéis hacer la base de una tienda, un blog, una agenda o una intranet.

No os asustéis por el número de archivos. Cada uno es corto y hace **una sola cosa**. Ese es precisamente el truco.

•
👌 Cómo vamos a trabajar



1
📘 Qué vamos a construir



2
🤔 La estructura de carpetas



3
👍 Paso 1 · Configuración y conexión



4
🔍 Paso 2 · La caja de herramientas



5
📘 Paso 3 · Las consultas, en su propio archivo



6
👌 Paso 4 · Cabecera y pie comunes



7
🚀 Paso 5 · Listado con buscador y paginación



8
👍 Paso 6 · Alta de cursos



9
🤔 Paso 7 · Edición con el formulario precargado



10
🐞 Paso 8 · Borrado seguro



11
⚠️ Paso 9 · Login contra la base de datos



12
😮 Probad vuestra aplicación



13
🚀 Ejercicios



14
👋 Lo que tenéis que llevaros de hoy





CLASE 16

👌 Cómo vamos a trabajar

Esta vez el proyecto tiene **su propia carpeta en Laragon** y su propia base de datos:

1. En Laragon, pulsad **Raíz** y cread la carpeta `academia-crud`.
2. Copiad dentro todo el contenido de la carpeta `proyecto/` de este repositorio.
3. Pulsad **Base de datos**, abrid la pestaña **Consulta**, pegad `base-de-datos.sql` y pulsad **F9**. Se crea `academia_crud`.
4. Pulsad **Recargar** y abrid `http://academia-crud.test`.

El usuario de prueba es `admin@academia.test` con la contraseña `secreto123`.



💡 Mi consejo: no os limitéis a copiar. Leed los archivos **en el orden de esta clase**, de uno en uno, y comprobad en el navegador lo que hace cada paso. Al final, intentad escribir alguno desde cero.



CLASE 16 · PUNTO 1

📘 Qué vamos a construir

Pantalla	Operación	Quién puede usarla
`index.php` · Catálogo con buscador y páginas	**R**ead	Cualquiera
`crear.php` · Nuevo curso	**C**reate	Solo usuarios con sesión iniciada
`editar.php?id=…` · Modificar un curso	**U**pdate	Solo usuarios con sesión iniciada
`borrar.php` · Borrar un curso (botón en el listado)	**D**elete	Solo usuarios con sesión iniciada
`login.php` y `logout.php` · Entrar y salir		Cualquiera

La base de datos es la de la clase 15 con dos cambios: hay más cursos (12, para que la paginación tenga sentido) y una tabla nueva, `usuarios`, con las contraseñas cifradas con `password_hash()`.



CLASE 16 · PUNTO 2

🤔 La estructura de carpetas

Esquema

```
academia-crud/
├── base-de-datos.sql          ← crea la base de datos (se ejecuta una vez en HeidiSQL)
├── config/
│   └── config.php             ← datos de conexión y ajustes: todo en un sitio
├── includes/                  ← piezas que usan todas las páginas
│   ├── inicio.php             ← carga todo lo de abajo y arranca la sesión
│   ├── conexion.php           ← conectar()
│   ├── funciones.php          ← e(), redirigir(), mensajes, CSRF, login…
│   ├── cursos.php             ← TODAS las consultas de la tabla cursos
│   ├── cabecera.php           ← <head>, menú y mensajes
│   ├── pie.php                ← <footer>
│   └── formulario-curso.php   ← el formulario, compartido por crear y editar
├── estilos.css
├── index.php                  ← listado
├── crear.php
├── editar.php
├── borrar.php
├── login.php
└── logout.php
```

La idea es la que visteis en la clase 14, llevada un poco más lejos: **cada cosa en su sitio**. Las páginas de la raíz son cortas porque se limitan a coordinar: leen lo que llega, llaman a las funciones y muestran el resultado. El SQL vive en `cursos.php`; el HTML común, en la cabecera y el pie.



💡 En la parte del MVC propio daremos un paso más: solo una carpeta `public/` será visible desde el navegador. Por ahora, así está perfecto.



CLASE 16 · PUNTO 3

👍 Paso 1 · Configuración y conexión

Todo lo que podría cambiar al pasar de vuestro ordenador a un servidor (datos de conexión, nombre de la web…) va en **un solo archivo** [1]:

config.php — php-paso-a-paso/clase16 — Visual Studio Code

config.php

```php
<?php
declare(strict_types=1);

const DB_HOST = "localhost";
const DB_NOMBRE = "academia_crud";
const DB_USUARIO = "root";
const DB_CLAVE = "";

const NOMBRE_WEB = "Academia Alpe";
const CURSOS_POR_PAGINA = 5;

date_default_timezone_set("Europe/Madrid");
```

📄 Archivos: `config/config.php` y `includes/conexion.php` (la función `conectar()` de la clase 15, sin cambios)



CLASE 16 · PUNTO 4

🔍 Paso 2 · La caja de herramientas

En `includes/funciones.php` están las funciones que usan todas las páginas. Algunas ya las conocéis (`e()`, `euros()`, el token CSRF de la clase 13). Os presento las nuevas:

Función	Qué hace
`redirigir("index.php")`	`header("Location: …")` **y** `exit`, juntos: así es imposible olvidar el `exit`
`guardarMensaje("Curso creado.")`	Guarda un mensaje en la sesión para mostrarlo en la **página siguiente**
`leerMensaje()`	Devuelve el mensaje guardado y lo borra, para que solo se vea una vez
`tokenCsrf()` / `comprobarCsrf($token)`	Crear y comprobar el token CSRF de los formularios
`usuarioConectado()`	Los datos del usuario de la sesión, o `null`
`requerirLogin()`	El **guardián**: si no hay sesión, os manda al login

Los mensajes *flash*

¿Recordáis el *Post/Redirect/Get*? Después de guardar, redirigimos… y entonces, ¿cómo le decimos al usuario que todo ha ido bien? En la clase 12 usábamos `?ok=1` en la URL. Ahora lo hacemos mejor: guardamos el mensaje en la **sesión** justo antes de redirigir, y la página siguiente lo muestra y lo borra. Como un pósit que se lee una vez y se tira.

ejemplo.php — php-paso-a-paso/clase16 — Visual Studio Code

ejemplo.php

```php
<?php
guardarMensaje("Curso «{$curso['titulo']}» creado.");
redirigir("index.php");
```

Un tipo de retorno nuevo: `never`

ejemplo.php — php-paso-a-paso/clase16 — Visual Studio Code

ejemplo.php

```php
<?php
function redirigir(string $destino): never
{
    header("Location: {$destino}");
    exit;
}
```

`never` significa «esta función **nunca** termina de forma normal»: siempre acaba con `exit` o con una excepción [2]. Es una forma de que PHP (y quien lea el código) sepa que después de `redirigir()` no se ejecuta nada más.

Todo cargado con una línea

Para no repetir cuatro `require_once` y el `session_start()` en cada página, `includes/inicio.php` lo hace por nosotros. Cada página empieza así:

ejemplo.php — php-paso-a-paso/clase16 — Visual Studio Code

ejemplo.php

```php
<?php
declare(strict_types=1);

require_once __DIR__ . "/includes/inicio.php";
```



CLASE 16 · PUNTO 5

📘 Paso 3 · Las consultas, en su propio archivo

Este es, para mí, el paso más importante de la clase. **Las páginas no escriben SQL.** Todas las consultas de la tabla `cursos` están en `includes/cursos.php`, cada una en una función con un nombre que dice lo que hace:

Función	Consulta
`contarCursos($pdo, $busqueda)`	`SELECT COUNT(*) … WHERE titulo LIKE …`
`obtenerCursos($pdo, $busqueda, $pagina, $porPagina)`	`SELECT … LIMIT … OFFSET …`
`obtenerCurso($pdo, $id)`	`SELECT … WHERE id = :id` → un array o `null`
`crearCurso($pdo, $curso)`	`INSERT …` → el id nuevo
`actualizarCurso($pdo, $id, $curso)`	`UPDATE … WHERE id = :id`
`borrarCurso($pdo, $id)`	`DELETE … WHERE id = :id` → `true` si ha borrado
`leerFormularioCurso($_POST)`	Recoge los datos del formulario, limpios y con su tipo
`validarCurso($curso)`	Devuelve el array de errores

¿Qué ganamos? Que `editar.php` se lee como una frase: «obtén el curso; si no existe, fuera; lee el formulario, valídalo y actualiza». Y que si mañana cambia la tabla, sabéis exactamente qué archivo tocar.



💡 Guardad esta idea, porque volverá: en la parte de POO, `cursos.php` se convertirá en una **clase** (el patrón *Repository*), y en Laravel será un **modelo** de Eloquent.



CLASE 16 · PUNTO 6

👌 Paso 4 · Cabecera y pie comunes

`includes/cabecera.php` pinta el `<head>`, el menú y el mensaje *flash* si lo hay. El menú cambia según haya sesión iniciada o no:

ejemplo.php — php-paso-a-paso/clase16 — Visual Studio Code

ejemplo.php

```php
<?php if ($usuario === null): ?>
    <a href="crear.php">➕ Nuevo curso</a>
    <span class="usuario">👤 <?= e($usuario["nombre"]) ?></span>
<?php else: ?>
    <a href="login.php">Iniciar sesión</a>
<?php endif; ?>
```

Fijaos en un detalle: el botón **Salir** es un pequeño formulario POST con su token CSRF, no un enlace. Cerrar la sesión también es una acción que cambia algo, y por eso, igual que el borrado, va por POST.



CLASE 16 · PUNTO 7

🚀 Paso 5 · Listado con buscador y paginación

Con 12 cursos podríamos mostrarlos todos. Pero con 50 000 productos, un `SELECT` sin límite traería **toda la tabla** y tumbaría el servidor. La solución es **paginar**: pedir solo los registros de la página que se está viendo [3].

Página	`LIMIT` (cuántos)	`OFFSET` (cuántos me salto)
1	5	0
2	5	5
3	5	10
n	5	(*n* − 1) × 5

index.php — php-paso-a-paso/clase16 — Visual Studio Code

index.php

```php
<?php
$consulta = $pdo->prepare(
    "SELECT id, titulo, horas, precio, plazas
     FROM cursos
     WHERE titulo LIKE :busqueda
     ORDER BY titulo
     LIMIT :limite OFFSET :desde"
);
$consulta->bindValue("busqueda", "%{$busqueda}%", PDO::PARAM_STR);
$consulta->bindValue("limite", $porPagina, PDO::PARAM_INT);
$consulta->bindValue("desde", ($pagina - 1) * $porPagina, PDO::PARAM_INT);
$consulta->execute();
```

¿Qué es eso de `bindValue()`? Otra forma de dar valor a los marcadores, uno a uno y **diciendo el tipo** [4]. En `LIMIT` y `OFFSET` os recomiendo usarla con `PDO::PARAM_INT`: si el número llegara como texto (`LIMIT '5'`), algunas versiones de MySQL dan error de sintaxis.

Y para saber cuántas páginas hay, contamos primero:

index.php — php-paso-a-paso/clase16 — Visual Studio Code

index.php

```php
<?php
$total = contarCursos($pdo, $busqueda);
$totalPaginas = max(1, (int) ceil($total / CURSOS_POR_PAGINA));
$pagina = min($pagina, $totalPaginas);   // si piden la página 99, mostramos la última
```

En los enlaces de las páginas mantenemos la búsqueda: `?q=php&pagina=2`. Si no, al cambiar de página se perdería el filtro.

📄 Archivo: `index.php`



CLASE 16 · PUNTO 8

👍 Paso 6 · Alta de cursos

`crear.php` sigue el orden de siempre y cabe en una pantalla:

crear.php — php-paso-a-paso/clase16 — Visual Studio Code

crear.php

```php
<?php
declare(strict_types=1);

require_once __DIR__ . "/includes/inicio.php";

$titulo = "Nuevo curso";
$errores = [];
$curso = ["titulo" => "", "horas" => 20, "precio" => 0, "plazas" => 20];

requerirLogin();
$pdo = conectar();
$esEnvio = $_SERVER["REQUEST_METHOD"] === "POST";

if ($esEnvio) {
    comprobarCsrf($_POST["csrf"] ?? "");
    $curso = leerFormularioCurso($_POST);
    $errores = validarCurso($curso);

    if (count($errores) === 0) {
        crearCurso($pdo, $curso);
        guardarMensaje("Curso «{$curso['titulo']}» creado.");
        redirigir("index.php");
    }
}

require __DIR__ . "/includes/cabecera.php";
?>

<h1>➕ Nuevo curso</h1>
<?php require __DIR__ . "/includes/formulario-curso.php"; ?>

<?php require __DIR__ . "/includes/pie.php"; ?>
```

📄 Archivo: `crear.php`

El formulario está en `includes/formulario-curso.php` y lo usan **crear y editar**. Solo necesita dos variables: `$curso` (lo que se muestra en cada campo) y `$errores`. Un único formulario: si mañana añadís un campo, lo añadís una vez.



⚠️ Los `required` y `min` del HTML ayudan al usuario, pero **la validación de verdad es** `validarCurso()`, en PHP. Recordad la clase 12: cualquiera puede saltarse el HTML.



CLASE 16 · PUNTO 9

🤔 Paso 7 · Edición con el formulario precargado

Editar es casi igual que crear, con una diferencia: al entrar, el formulario aparece **ya relleno** con los datos guardados. Eso se llama **precargar**.

editar.php — php-paso-a-paso/clase16 — Visual Studio Code

editar.php

```php
<?php
$id = (int) ($_GET["id"] ?? 0);

$cursoGuardado = obtenerCurso($pdo, $id);       // 1. ¿existe?
if ($cursoGuardado === null) {
    guardarMensaje("Ese curso no existe.", "error");
    redirigir("index.php");
}

$curso = $cursoGuardado;                         // 2. precarga

if ($esEnvio) {                                  // 3. al enviar, lo que ha escrito el
usuario
    comprobarCsrf($_POST["csrf"] ?? "");
    $curso = leerFormularioCurso($_POST);
    $errores = validarCurso($curso);
    if (count($errores) === 0) {
        actualizarCurso($pdo, $id, $curso);
        guardarMensaje("Curso «{$curso['titulo']}» actualizado.");
        redirigir("index.php");
    }
}
```

¿Por qué guardamos `$cursoGuardado` aparte? Para que el título de la página siga mostrando el nombre original («Editar PHP 8 desde cero») aunque el usuario lo esté cambiando en el formulario.

📄 Archivo: `editar.php`



CLASE 16 · PUNTO 10

🐞 Paso 8 · Borrado seguro

Ya os lo he dicho en las clases 13 y 15, pero hoy lo aplicamos todo junto. Un borrado seguro tiene **cuatro capas**:

Capa	Cómo	Contra qué protege
1. Solo POST	`borrar.php` responde **405** a cualquier petición GET	Enlaces que alguien (o algo) visita sin querer
2. Confirmación	`onsubmit="return confirm('¿Seguro…?')"`	Un clic despistado
3. Token CSRF	`comprobarCsrf($_POST["csrf"])`	Formularios trampa desde otras webs
4. Sesión	`requerirLogin()`	Visitantes sin permiso

Y una quinta, que pone la base de datos: **la clave foránea**. Un curso que tiene alumnos **no se puede borrar**: MySQL lanza el error **1451** y lo convertimos en un mensaje amable.

borrar.php — php-paso-a-paso/clase16 — Visual Studio Code

borrar.php

```php
<?php
try {
    if (borrarCurso($pdo, $id)) {
        guardarMensaje("Curso borrado.");
    } else {
        guardarMensaje("Ese curso ya no existe.", "error");
    }
} catch (PDOException $e) {
    if (($e->errorInfo[1] ?? 0) == 1451) {
        throw $e;
    }
    guardarMensaje("No se puede borrar: el curso tiene alumnos matriculados.", "error");
}

redirigir("index.php");
```

📄 Archivo: `borrar.php` . Probad a borrar «PHP 8 desde cero», que tiene alumnos.



CLASE 16 · PUNTO 11

⚠️ Paso 9 · Login contra la base de datos

En la clase 13 los usuarios estaban en un array. Ahora vienen de la tabla `usuarios`, con la contraseña cifrada en la columna `clave_hash` [5]:

login.php — php-paso-a-paso/clase16 — Visual Studio Code

login.php

```php
<?php
$usuario = buscarUsuarioPorEmail($pdo, $email);

if ($usuario !== null && password_verify($clave, $usuario["clave_hash"])) {
    session_regenerate_id(true);
    $_SESSION["usuario"] = ["id" => $usuario["id"], "nombre" => $usuario["nombre"]];
    guardarMensaje("¡Hola, {$usuario['nombre']}!");
    redirigir("index.php");
}

$error = "Email o contraseña incorrectos.";
```

Tres detalles que ya conocéis y que no podéis olvidar:

- `password_verify()` compara la contraseña escrita con el hash. **Nunca** `===`.
- El mismo mensaje de error tanto si falla el email como si falla la contraseña.
- En la sesión guardamos **solo** el id y el nombre. El hash de la contraseña no pinta nada ahí.

Y en cada página privada, el guardián en una línea:

login.php — php-paso-a-paso/clase16 — Visual Studio Code

login.php

```php
<?php
requerirLogin();
```

📄 Archivos: `login.php` y `logout.php`



CLASE 16 · PUNTO 12

😮 Probad vuestra aplicación

Antes de dar un proyecto por terminado, se prueba **todo**, también lo que debería fallar. Esta es mi lista:

Haced esto	Debéis obtener
Abrid el catálogo sin iniciar sesión	El listado, sin botones de editar ni borrar
Buscad «php»	1 curso, y el filtro se mantiene al cambiar de página
Abrid `?pagina=99`	La última página, sin errores
Abrid `crear.php` sin sesión	Os manda al login con un aviso
Entrad con una contraseña incorrecta	«Email o contraseña incorrectos»
Entrad con `admin@academia.test` / `secreto123`	El listado, ahora con acciones y «👤 Administración»
Cread un curso con título «AB» y horas 0	Los errores junto a cada campo, sin perder lo escrito
Cread un curso correcto	Vuelta al listado con el mensaje verde
Editad un curso	El formulario aparece relleno; al guardar, mensaje verde
Abrid `editar.php?id=999`	Vuelta al listado con «Ese curso no existe»
Borrad «PHP 8 desde cero»	«No se puede borrar: el curso tiene alumnos»
Borrad el curso que habéis creado	«Curso borrado»
Abrid `borrar.php?id=2` en la barra de direcciones	Error 405: por GET no se borra nada
Pulsad «Salir»	Vuelta al catálogo sin sesión



Las soluciones son páginas nuevas para **copiar dentro de la carpeta del proyecto**, junto a `index.php`.

- ☐ **01 · Ficha del curso.** `ver.php?id=…` con los datos del curso y la lista de sus alumnos. Haced que el título de cada curso del listado enlace a su ficha. → `ver.php`
- ☐ **02 · Ordenar el listado.** Un `<select>` para ordenar por título, precio (de menor a mayor y al revés) u horas. Cuidado: los nombres de columna **no** pueden ir en un marcador (`ORDER BY :orden` no funciona). Pista: una lista blanca con los valores permitidos. → `index-ordenado.php`
- ☐ **03 · Registro.** `registro.php` para crear cuentas nuevas: nombre, email y contraseña repetida, guardada con `password_hash()`. Si el email ya existe, avisad. → `registro.php`
- ☐ **04 · Exportar a CSV.** `exportar.php` descarga el catálogo como archivo CSV que Excel abra con las tildes bien. Pista: las cabeceras `Content-Type` y `Content-Disposition`, y `fputcsv()` sobre `php://output`. → `exportar.php`



🏆 **Reto final:** haced el CRUD completo de la tabla `alumnos`: listado con el título de su curso (`INNER JOIN`), alta y edición con un `<select>` de cursos y borrado seguro. Tenéis todas las piezas: es repetir el proceso de hoy con otra tabla. Si lo conseguís sin mirar, este bloque está superado. 💪



CLASE 16 · PUNTO 14

👋 Lo que tenéis que llevaros de hoy

Idea	En una frase
CRUD	Crear, leer, actualizar y borrar: la base de casi cualquier aplicación
Carpetas	Configuración, piezas comunes y páginas cortas que coordinan
SQL en un solo archivo	Las páginas llaman a funciones; el SQL vive en `cursos.php`
`redirigir()`	`header()` y `exit` juntos; tipo de retorno `never`
Mensajes *flash*	Se guardan en la sesión, se muestran una vez y se borran
Paginación	`LIMIT` y `OFFSET` con `bindValue(..., PDO::PARAM_INT)`
Un formulario para dos	Crear y editar comparten `formulario-curso.php`
Precarga	Al editar, el formulario empieza con los datos guardados
Borrado seguro	POST, confirmación, token CSRF, sesión y clave foránea
Login con base de datos	`password_verify()` contra `clave_hash`; en la sesión, solo id y nombre

📚 Fuentes y documentación oficial

1. PHP: The Right Way · Bases de datos
2. Manual de PHP: declaraciones de tipo (never)
3. MySQL 8.0 Reference Manual: SELECT Statement (LIMIT)
4. Manual de PHP: PDOStatement::bindValue
5. Manual de PHP: password_verify